



Certificado profesional de Microsoft Python Development

Launch your career as a Python Developer.

Learn in-demand Python skills and become a job-ready developer in less than 4 months. No degree or experience required.



Instructor: [Microsoft](#)

[Ir al curso](#)

Ya estás inscrito

73.593 ya inscrito

Incluido con [Coursera Plus](#) • [Obtener más información](#)



[Python Programming Fundamentals](#)

CURSO 1 • 25 horas



[Data Analysis and Visualization with Python](#)

CURSO 2 • 22 horas



[Automation and Scripting with Python](#)

CURSO 3 • 25 horas



[Web Development with Python](#)

CURSO 4 • 23 horas



[Advanced Python Development Techniques](#)

CURSO 5 • 22 horas



[Project Development in Python](#)

CURSO 6 • 20 horas



Obtén un certificado profesional

Agregue esta credencial a su perfil de LinkedIn, currículum o CV. Compártala en redes sociales y en su evaluación de desempeño.

Plan PLATA 2026



Sentencias de Decisión y Selección en Programación

En el mundo del desarrollo de software, el código debe responder a entornos cambiantes y entradas diversas.

Las **sentencias de decisión y selección**

son la base de la lógica programática:

permiten al código **evaluar información y elegir acciones**,

de forma muy similar al pensamiento humano.

LÓGICA DE PROGRAMACIÓN

PYTHON

CONTROL DE FLUJO

¿Qué veremos hoy?

1

Sentencias de decisión

El bloque **if** y el control del flujo del programa basado en condiciones verdaderas o falsas.

2

Sentencias de selección

Estructuras **if-elif-else** para manejar múltiples caminos de ejecución.

3

Condiciones anidadas y lógica booleana

Operadores **and**, **or** y **not** para construir condiciones complejas.

4

Bucles

¿Qué son los bucles y para qué sirven?
bucles **For** y **While**, Bucles anidados

5

Seguimiento y Depuración

guía práctica y didáctica para comprender cómo
Python ejecuta tu código

6

Listas

as listas son la herramienta fundamental de
Python para



¿Qué es una Sentencia de Decisión?

Toda decisión en programación se basa en una pregunta fundamental:

¿la condición es verdadera o falsa?

Las sentencias de decisión evalúan una condición y determinan qué bloque de código se ejecutará a continuación.

La pregunta clave

- ❖ Cada **if** plantea una condición.
 - Si el resultado es **True**,
 - el código dentro del bloque se ejecuta.
 - Si es **False**,
 - el programa lo omite y continúa.

Ejemplo práctico

```
if order_total >= 50:  
    print("¡Envío gratuito!")  
# Si order_total es 60 →  
imprime  
# Si order_total es 30 → no  
hace nada
```

Aquí, el mensaje solo aparece si el total del pedido supera los 50. En caso contrario, el programa continúa sin hacer nada adicional.

Sentencias de Selección: Más de Dos Caminos

Cuando hay más de dos posibilidades, usamos **if-elif-else**. Python evalúa las condiciones de arriba hacia abajo y ejecuta **solo el primer bloque verdadero**. Esto nos permite modelar decisiones complejas con precisión.

1

if

Primera condición evaluada. Si es verdadera, se ejecuta y se salta el resto.

2

elif

Se evalúa solo si las anteriores fueron falsas. Puede haber tantos **elif** como necesitemos.

3

else

Bloque final que se ejecuta si ninguna condición anterior fue verdadera. Actúa como "en todos los demás casos".

```
if points >= 1000:
    print("¡Estado Platino! Disfruta de beneficios exclusivos.")
elif points >= 500:
    print("¡Eres miembro Gold! Gracias por tu fidelidad.")
elif points >= 100:
    print("Bienvenido al nivel Plata. Empieza a ganar recompensas.")
else:
    print("Eres miembro Bronce. ¡Sigue comprando para subir de nivel!")
```


Condiciones Anidadas

Las **condiciones anidadas** permiten colocar un bloque **if** dentro de otro, creando estructuras jerárquicas de decisión.

Son útiles cuando una decisión depende de que se haya cumplido una condición previa.

¿Cómo funcionan?

Imagina que primero compruebas si un usuario está registrado, y solo si lo está, verificas si tiene permisos de administrador.

Eso es exactamente lo que permiten los **if anidados**.

```
if user_logged_in:
    if user_is_admin:
        print("Acceso al panel de administración.")
    else:
        print("Acceso estándar.")
else:
    print("Por favor, inicia sesión.")
```

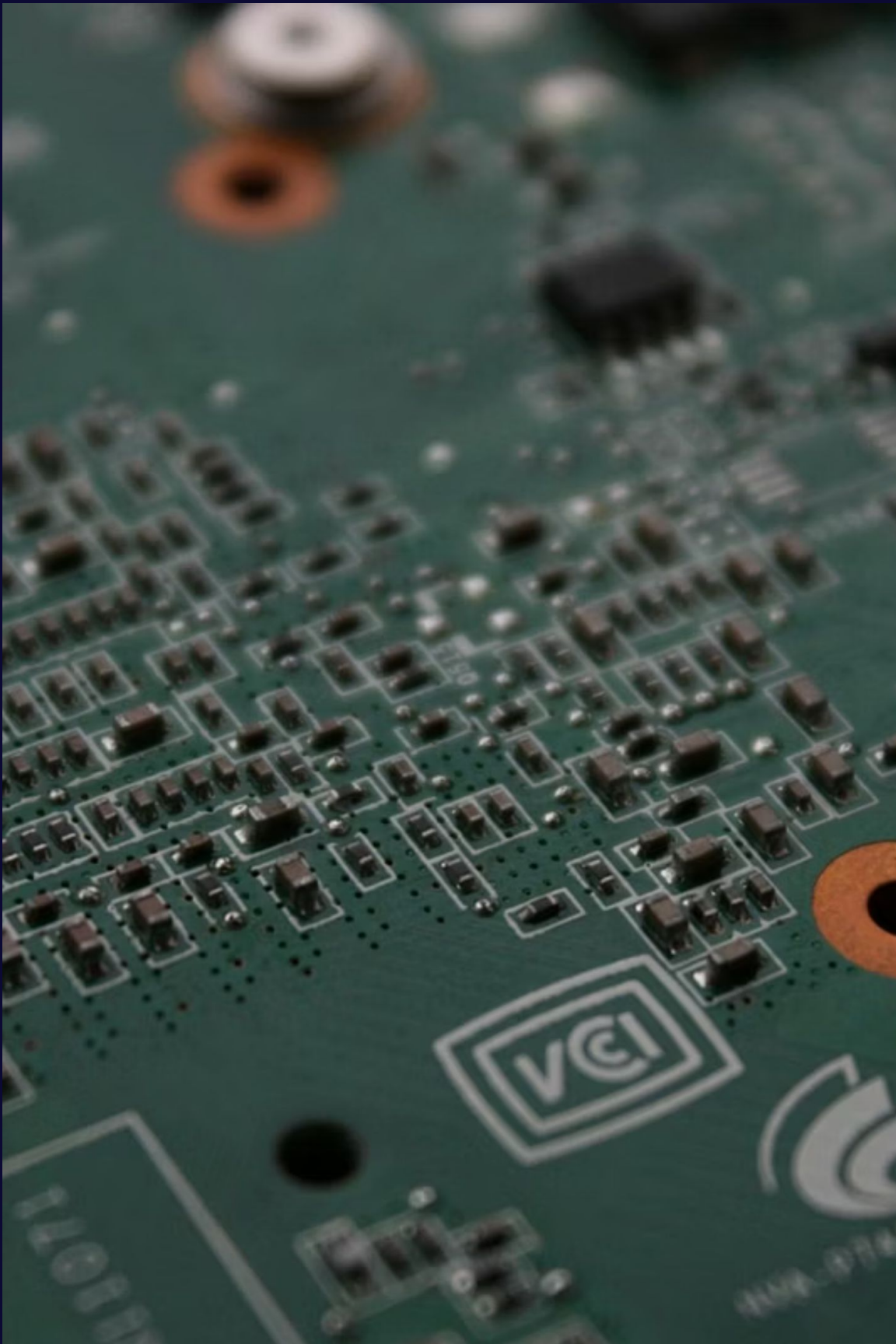
⚠️ Precaución con la profundidad

Aunque los anidamientos son poderosos, anidar demasiados niveles puede hacer el código **difícil de leer y mantener**.

Como norma general, evita más de tres niveles de anidamiento.

Considera refactorizar usando funciones si la lógica se vuelve compleja.

💡 **Regla de oro:** Si tienes más de 3 niveles de indentación, es señal de que debes dividir el código en funciones más pequeñas.



Lógica Booleana: and, or, not

Los **operadores booleanos** permiten combinar múltiples condiciones en una sola expresión, reduciendo la necesidad de anidamientos y haciendo el código más expresivo y conciso.

and (y)

Ambas condiciones deben ser verdaderas para que el resultado sea **True**.

```
if age >= 18 and has_id:  
    print("Acceso permitido")
```

or (o)

Basta con que **una** de las condiciones sea verdadera para que el resultado sea **True**.

```
if is_member or has_coupon:  
    print("Descuento aplicado")
```

not (no)

Invierte el valor de la condición: convierte **True** en **False** y viceversa.

```
if not is_banned:  
    print("Bienvenido")
```



Importante: Las variables booleanas deben definirse antes de usarlas.

Por ejemplo: `is_member = True`

Aplicaciones Reales de las Sentencias Condicionales

Las estructuras condicionales no son solo ejercicios académicos. Están presentes en los sistemas más sofisticados del mundo real, tomando decisiones críticas en tiempo real cada segundo.



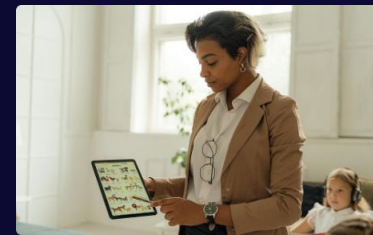
Vehículos Autónomos

Toman decisiones en tiempo real: frenar, girar o acelerar según las condiciones del entorno.



Detección de Fraude

Analizan transacciones y aplican reglas condicionales para identificar comportamientos sospechosos.



Aprendizaje Personalizado

Plataformas educativas adaptan el contenido según el rendimiento y progreso de cada estudiante.



Domótica

Sistemas de hogar inteligente activan luces, temperaturas y alarmas según condiciones predefinidas.

Buenas Prácticas: Código Limpio y Mantenable

El uso excesivo y desordenado de estructuras **if-else** puede generar código difícil de leer, propenso a errores y complejo de mantener. Seguir estas buenas prácticas marcará la diferencia en la calidad de tu código.

⚠ Señales de alarma

- Demasiados niveles de anidamiento
- Condiciones largas y difíciles de leer
- Repetición de lógica similar

Variables con nombres como `x`, `flag` o `temp`

✅ Buenas prácticas

Modularizar en funciones

Extrae bloques de lógica condicional en funciones con nombres descriptivos.

Usar nombres descriptivos

Variables como `is_premium_user` o `has_valid_license` hacen el código autoexplicativo.

Escribir comentarios claros

Explica el *por qué* de una condición, no solo el *qué* hace.

Optimización: Más Allá del if-else

Cuando tenemos muchas opciones posibles, el encadenamiento de **if-elif** puede volverse engorroso. Python ofrece alternativas más elegantes y eficientes para estos escenarios.

Diccionarios como tabla de decisión

En lugar de múltiples **elif**, un diccionario mapea directamente cada caso a su resultado. Es más limpio, rápido y fácil de ampliar.

```
shipping_rates = {  
    "ES": 5,  
    "EU": 10,  
    "US": 15  
}  
country = "ES"  
rate = shipping_rates.get(country, 20)  
print(f"Coste de envío: {rate}€")
```

Librerías para grandes volúmenes

Cuando trabajamos con miles o millones de datos, las librerías especializadas permiten aplicar condiciones de forma vectorizada, sin bucles lentos.

NumPy

Operaciones condicionales sobre arrays numéricos con `np.where()`.

Pandas

Filtrado y clasificación condicional de tablas de datos con `df[df['col'] > valor]`.

Resumen: Lo Esencial de las Sentencias Condicionales



El código analiza y decide

Las estructuras condicionales dan inteligencia al programa: evalúan el contexto y eligen la acción más apropiada.



De simple a complejo

Desde un solo **if** hasta cadenas de **elif**, condiciones anidadas y operadores booleanos para lógica sofisticada.



La calidad importa

Un buen uso de condicionales produce código claro, eficiente y mantenible. Las malas prácticas generan deuda técnica.



Clave para el desarrollo profesional

Dominar estos conceptos es el primer paso para construir software dinámico, robusto y de calidad profesional.



👉 Recuerda: Las sentencias de decisión y selección son el corazón de cualquier programa que reaccione al mundo real. Practícalas con consistencia y notarás una mejora inmediata en la calidad de tu código.

Pregunta de consolidación

Pon a Prueba Tu Comprensión

Estás desarrollando un programa Python para **automatizar la clasificación de tickets de soporte al cliente** según su nivel de urgencia. ¿Cuál de los siguientes escenarios demuestra **mejor** el uso de sentencias condicionales en este contexto?

A) Mostrar un mensaje de bienvenida

Mostrar un mensaje de bienvenida al agente cuando se conecta al sistema.

B) Calcular el tiempo medio de respuesta

Cálculo del tiempo medio de respuesta para todas las solicitudes, independientemente de su urgencia.

C) Almacenar todos los tickets en una lista

Guardar todos los tickets en una lista, independientemente de su urgencia o contenido.

D) Asignar tickets urgentes a cola prioritaria

Asignar tickets etiquetados como "urgentes" a una cola prioritaria y notificar inmediatamente al equipo correspondiente.

Respuesta correcta

La Respuesta es la Opción D

¿Por qué la opción D?

Esta opción es la única que demuestra el uso genuino de sentencias condicionales: el programa **evalúa una condición** (nivel de urgencia del ticket) y **desencadena una acción específica** en función de ese valor. Exactamente esto es lo que hacen las sentencias condicionales: **analizar, decidir y actuar** de manera diferente según el contexto.

¿Por qué las demás son incorrectas?

A: Solo ejecuta código sin evaluar ninguna condición.

B: Es un cálculo que se aplica igual a todos los casos, sin decisión.

C: Almacena datos sin diferenciar ni tomar ninguna decisión.



Las sentencias condicionales brillan cuando el programa debe **comportarse diferente** según el estado de los datos.

Bucles y Sentencias Condicionales en Python

Dos herramientas fundamentales que todo programador necesita dominar:

- ❖ Los **bucles** para automatizar la repetición.
- ❖ Las **sentencias condicionales** para tomar decisiones inteligentes.

Juntas, forman el corazón de cualquier programa dinámico y eficiente.



¿Qué son los bucles y para qué sirven?

El problema sin bucles

Imagina que quieres imprimir los números del 1 al 10.
Sin bucles, necesitarías escribir 10 líneas de código manualmente. Ahora imagina hacerlo con 1.000 números...

La solución con bucles

Un bucle permite ejecutar un bloque de código repetidamente con una sola instrucción.

Python ofrece dos tipos principales:

for → itera sobre una **secuencia** conocida

while → repite mientras una **condición** sea verdadera

2

Tipos de bucles

for y while

10X

Menos código

Mayor eficiencia

∞

Posibilidades

De repetición controlada

El Bucle for: Iteración sobre Secuencias

El bucle **for** es ideal cuando conoces de antemano cuántas veces necesitas repetir una acción, o cuando trabajas con una colección de elementos.

Su sintaxis es clara y expresiva.

```
for i in range(1, 11):  
    print(i)  
# Imprime: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
```

La función range()

`range(stop)` → de 0 a stop-1

`range(start, stop)` →
de start a stop-1

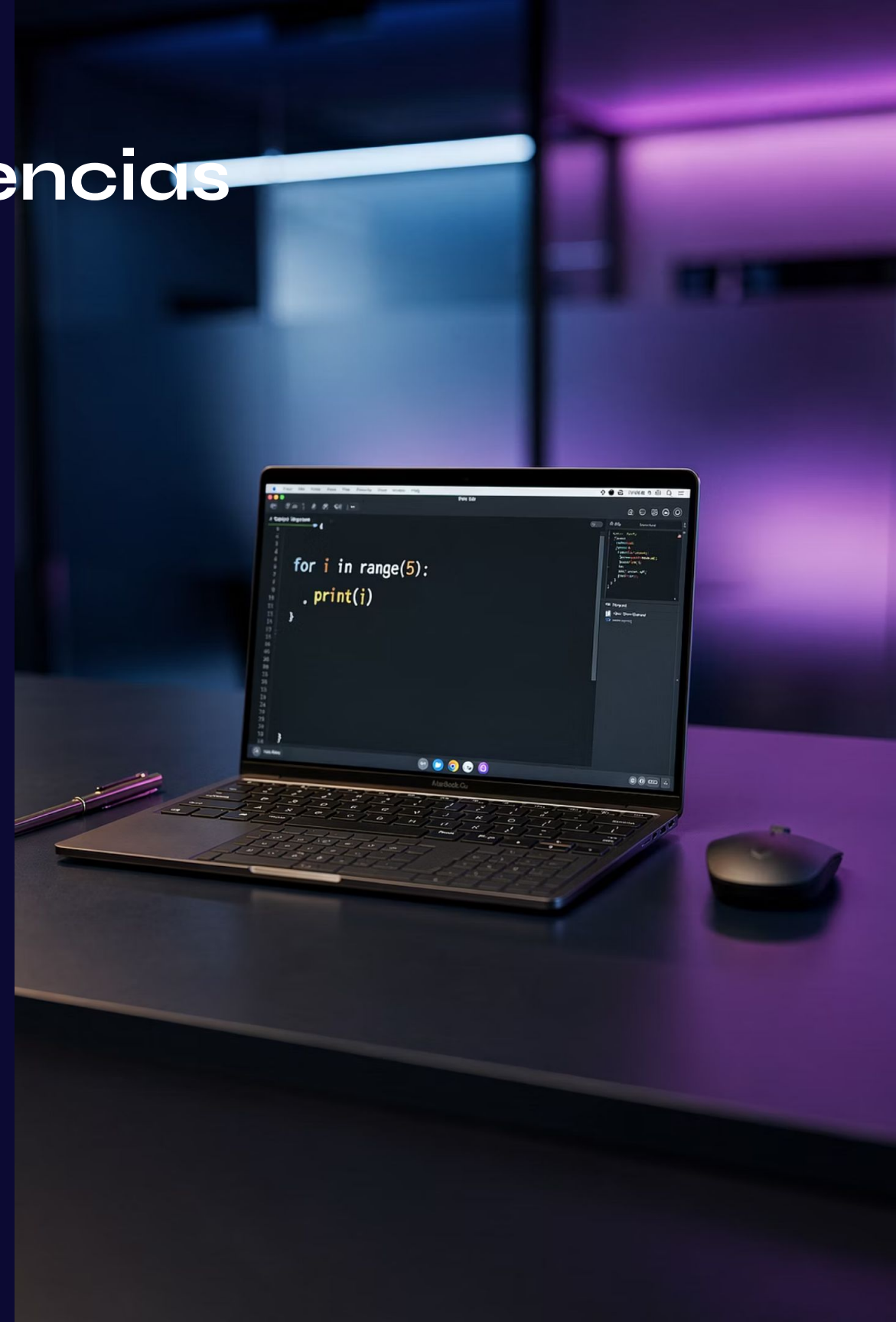
`range(start, stop, step)` →
con saltos personalizados

Ejemplo:

`range(1, 10, 2)` → 1, 3, 5, 7, 9

¿Cuándo usar for?

- Recorrer listas y tuplas
- Iterar sobre cadenas de texto
- Trabajar con rangos numéricos
- Procesar colecciones de datos



El Bucle `while`: Condición Dinámica

El bucle **while** ejecuta su bloque de código *mientras* una condición sea verdadera. Es más flexible que **for** cuando no sabes cuántas iteraciones necesitarás. Es esencial para esperar entradas del usuario o manejar estados cambiantes.

```
number = 1

while number <= 10:

    print(number)

    number = number + 1
```

Este ejemplo imprime números del 1 al 10. El bucle se detiene cuando `number` supera 10.

Ejemplo real: validación de entrada del usuario.

```
valid_input = False

while not valid_input:

    user_input = int(input("Introduce un número > 0:

    "))

    if user_input > 0:

        valid_input = True

    else:

        print("Entrada inválida. Inténtalo de nuevo.")
```



⚠ Cuidado con los bucles infinitos:

Si la condición del `while` nunca se vuelve falsa, el programa se quedará bloqueado.

Asegúrate siempre de que el estado cambia dentro del bucle.

Bucles Anidados: Estructuras Complejas

Un bucle anidado es un bucle dentro de otro bucle. Son útiles para trabajar con matrices, tablas o cualquier estructura de datos bidimensional. El bucle interior se ejecuta completamente por cada iteración del bucle exterior.

```
for i in range(1, 4):  
    for j in range(1, 4):  
        print(i, "*", j, "=", i * j, end="\t")  
    print()
```

Salida:

```
1 * 1 = 1    1 * 2 = 2    1 * 3 = 3  
2 * 1 = 2    2 * 2 = 4    2 * 3 = 6  
3 * 1 = 3    3 * 2 = 6    3 * 3 = 9
```

Desafío

Números divisibles por 3 y 4 hasta 50

Código

```
for n in range(51):  
    if n%3==0 and n%4==0:  
        print(n)
```

Salida

0, 12, 24, 36, 48

  **Evita el exceso de anidamiento:** Más de dos niveles de bucles anidados puede dificultar la lectura y el mantenimiento. Considera refactorizar en funciones si el código se vuelve muy complejo.

Listas y Bucles For : Una Combinación Esencial

Las listas son una de las estructuras de datos más utilizadas en Python. Combinarlas con bucles permite procesar colecciones de información de forma elegante y eficiente.

Operaciones básicas con listas

```
fruits = ["apple", "banana", "cherry"]  
first_fruit = fruits[0]    # Acceso por índice  
len(fruits)                # Longitud: 3  
fruits.append("date")      # Añadir elemento
```

Iteración: forma recomendada

```
students = ["Alice", "Bob", "Charlie"]  
  
for student in students:  
    print("Hello,", student)
```

Buenas Prácticas y Errores Comunes

✓ Recomendaciones

Usa **for** cuando conoces el número de iteraciones
y **while** para condiciones dinámicas

Emplea **nombres descriptivos** para variables de control:
`alumnos` en lugar de `x`

Mantén una **indentación consistente** de 4 espacios como dicta
la PEP 8

Considera **comprensiones de lista** como alternativa elegante a
bucles simples

✗ Errores a evitar

Bucles infinitos

Olvidar modificar la variable de control en un `while`

Anidamiento excesivo

Más de 2 niveles dificulta la lectura; refactoriza en funciones

Condiciones complejas

Simplifica con variables auxiliares de nombre descriptivo

Seguimiento y Depuración de Código en Python

Una guía práctica y didáctica para comprender cómo Python ejecuta tu código, identificar errores comunes y convertirte en un programador más eficiente y seguro.

DEPURACIÓN

PYTHON

DESARROLLO

```
def get_row_and_columns_for_second_matrix():
    # Prompt user for row and columns for second matrix.
    print("\nEnter row and columns for second matrix.")
    r2 = int(input("Enter row for matrix 2: "))
    c2 = int(input("Enter column for matrix 2: "))

    # Prompt user for elements of first matrix.
    print("\nEnter elements of matrix 1:")
    for i in range(r2):
        for j in range(c2):
            print("Enter element a", i + 1, j + 1, ": ", end="")
            a = input()
            matrix1[i][j] = a
            i = i + 1
            j = j + 1
        j = 0
    i = 0
    return matrix1

def get_row_and_columns_for_second_matrix():
    # Prompt user for row and columns for second matrix.
    print("\nEnter row and columns for second matrix.")
    r2 = int(input("Enter row for matrix 2: "))
    c2 = int(input("Enter column for matrix 2: "))

    # Prompt user for elements of first matrix.
    print("\nEnter elements of matrix 1:")
    for i in range(r2):
        for j in range(c2):
            print("Enter element a", i + 1, j + 1, ": ", end="")
            a = input()
            matrix1[i][j] = a
            i = i + 1
            j = j + 1
        j = 0
    i = 0
    return matrix1
```

Seguimiento de la Ejecución de Código

¿Qué es el tracing?

El **seguimiento (tracing)** consiste en analizar cómo se ejecuta un programa instrucción por instrucción. Permite visualizar el flujo real del código, no solo el resultado final.

¿Quién lo necesita?

Es una técnica esencial tanto para **principiantes** que aprenden a programar como para **desarrolladores experimentados** que depuran sistemas complejos o revisan código ajeno.

¿Para qué sirve el rastreo de código?

El rastreo no es solo una herramienta de depuración: es una habilidad fundamental que mejora tu comprensión del código y tu eficiencia como desarrollador.



Depuración

Identifica la línea exacta donde ocurre un error, eliminando conjeturas y ahorrando tiempo.



Comprensión

Ayuda a entender la lógica de código complejo, especialmente cuando trabajas con librerías o proyectos heredados.



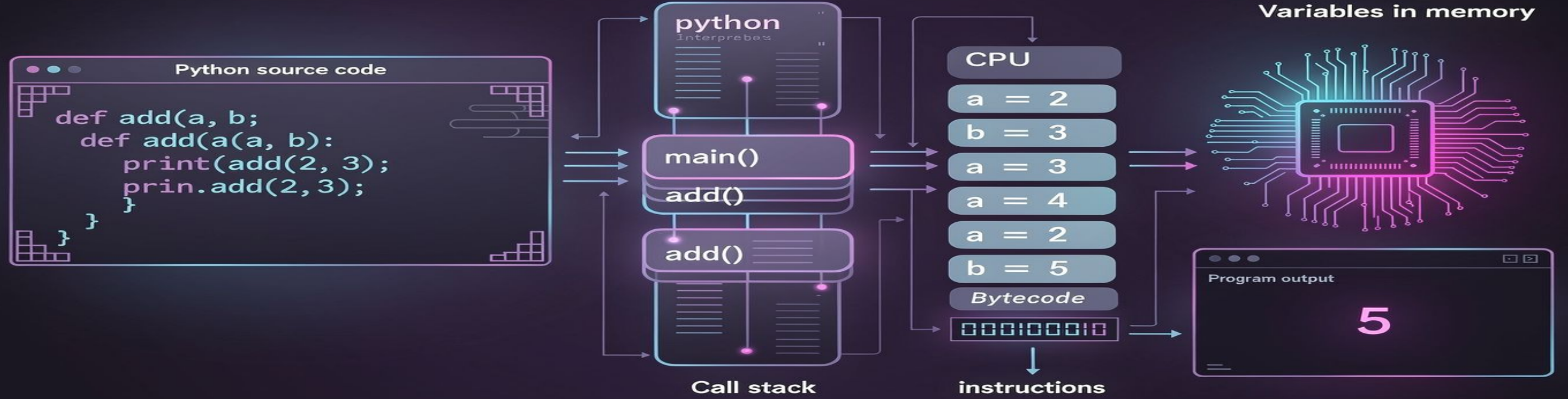
Colaboración

Facilita trabajar con código escrito por otros miembros del equipo, reduciendo la curva de aprendizaje.



Optimización

Permite detectar ineficiencias y cuellos de botella antes de que afecten al rendimiento en producción.



Componentes Clave del Seguimiento

🧩 Intérprete

Ejecuta el código **línea por línea**, traduciendo cada instrucción Python a operaciones que la máquina puede procesar. Este comportamiento secuencial es fundamental para entender el flujo de ejecución.

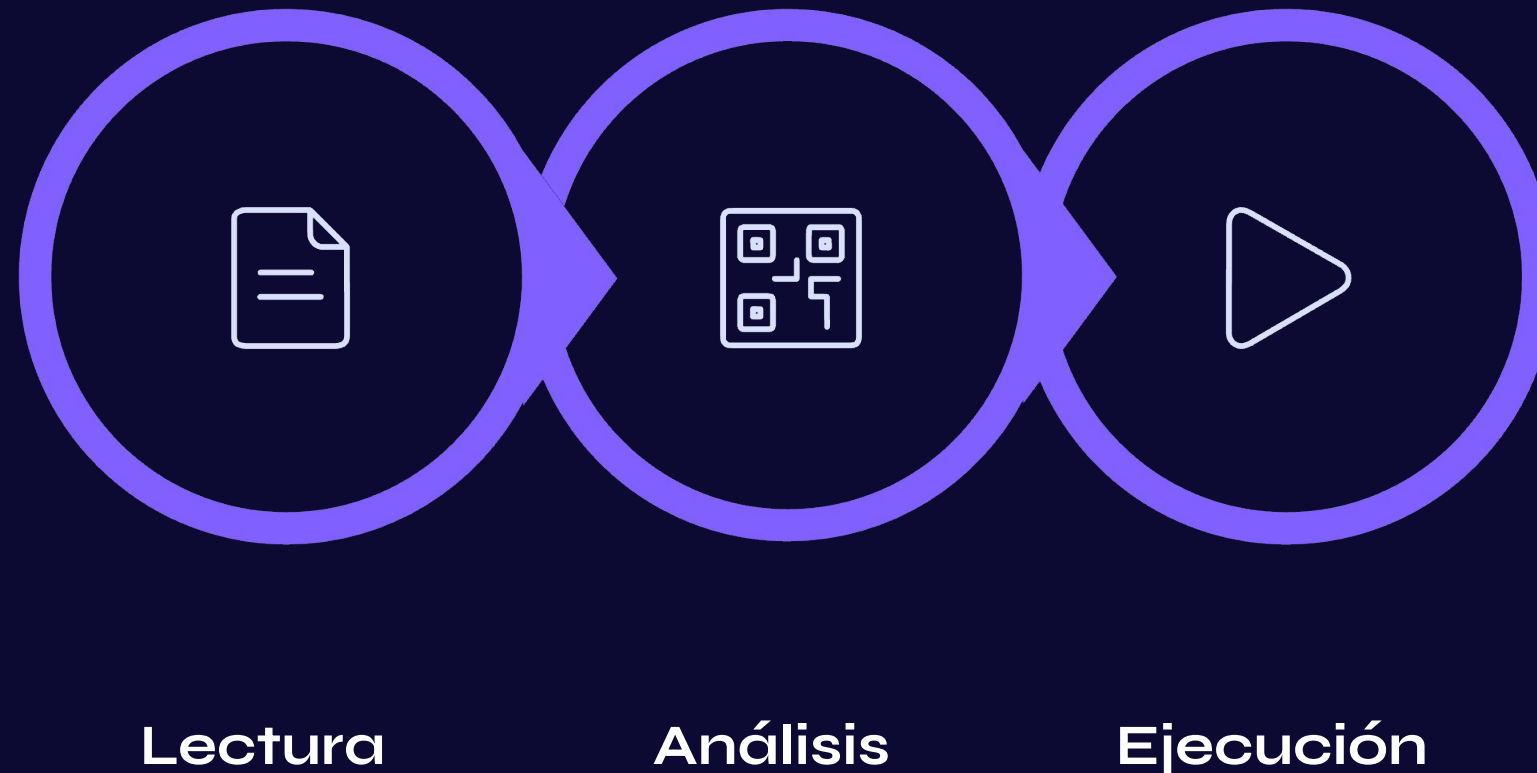
📖 Pila de llamadas (Call Stack)

Registra todas las funciones activas en cada momento. Cada llamada agrega un "**marco**" a la pila, permitiendo rastrear el flujo del programa, detectar errores de recursión y controlar los valores de retorno.

📦 Variables

Almacenan datos como números, texto, listas o cualquier objeto. El rastreo permite observar cómo **cambian sus valores** a lo largo de la ejecución, revelando comportamientos inesperados.

Cómo Funciona el Intérprete de Python



El intérprete de Python procesa el código de forma **secuencial**: lee, analiza y ejecuta cada instrucción en orden. Este comportamiento línea a línea es la razón por la que un error en la línea 5 puede detener por completo la ejecución, aunque el resto del código sea correcto.

Herramienta Esencial: `print()`

La función `print()` es la herramienta de rastreo más inmediata y accesible en Python.

Con solo una línea de código puedes inspeccionar el estado de tu programa en cualquier momento de su ejecución.

Ver valores de variables

Muestra el contenido de cualquier variable en tiempo real.

Seguir el flujo de ejecución

Confirma qué bloques de código se están ejecutando realmente.

Detectar errores ocultos

Revela valores inesperados antes de que provoquen un fallo.

```
# Ejemplo de rastreo con print()
def calcular_total(precio, cantidad):
    print("precio =", precio)
    print("cantidad =", cantidad)
    total = precio * cantidad
    print("total =", total)
    return total
```

```
calcular_total(5.99, 3)
```

Salida:

```
precio = 5.99
```

```
cantidad = 3
```

```
total = 17.97
```


Ejemplo 2: Funciones Anidadas y Ámbito

```
def outer(x):  
    y = x + 5  
    def inner():  
        z = y * 2  
        print("z =", z)  
    inner()  
    print("y =", y)
```

```
outer(10)
```

```
# Salida:
```

```
# z = 30
```

```
# y = 15
```

Traza de ejecución

1 `outer(10) → x = 10`

Se inicia la ejecución de la función exterior con el argumento 10.

2 `y = 10 + 5 = 15`

Se calcula y almacena el valor de `y` en el ámbito de `outer`.

3 `inner()` accede a `y`

La función anidada lee `y = 15` y calcula `z = 30`.

4 Retorno a `outer()`

Tras ejecutar `inner()`, continúa el flujo en `outer()` e imprime `y = 15`.

💡 **Concepto clave:** La función interna `inner()` puede acceder a la variable `y` definida en `outer()`. Esto se llama **ámbito léxico** o *closure*.

Pregunta Práctica: ¿Cómo Depurar un Cálculo?

Escenario: Tienes un programa que calcula el coste total de artículos multiplicando el precio por artículo por el número de artículos. Quieres entender exactamente cómo funciona este cálculo paso a paso.

1

Leer las instrucciones

Revisar la documentación para entender cómo *debería* funcionar el programa. Útil, pero no muestra la ejecución real.

2

Añadir líneas de `print()`

Insertar `print()` en distintas etapas permite ver el precio, la cantidad y el coste total en tiempo real, revelando exactamente cómo se llega al resultado.

3

Usar un depurador

Pausar el programa con un debugger es otra opción válida, aunque requiere herramientas adicionales y más configuración inicial.

4

Consultar el historial

Ver registros anteriores no ayuda a entender el cálculo actual en detalle ni a observar los valores intermedios.



PARTE 2

Errores Comunes en la Ejecución de Código Python

Incluso los programadores experimentados cometen errores de ejecución. Conocerlos con antelación te permite escribir código más **limpio, fiable y eficiente** desde el principio.

Conceptos Avanzados a Tener en Cuenta



Importaciones Circulares

Ocurren cuando dos módulos se importan mutuamente. Python no puede resolver la dependencia y lanza un `ImportError`. La solución pasa por reorganizar el código o usar importaciones locales dentro de funciones.



Gestión de Memoria

El uso ineficiente de memoria puede generar fallos graves en producción. Utiliza estructuras de datos adecuadas, evita referencias circulares y aprovecha el módulo `gc` para gestión manual cuando sea necesario.



Pruebas y Depuración

A medida que el software crece, las pruebas manuales son insuficientes. Las **pruebas automatizadas** con herramientas como `pytest` y los depuradores integrados en IDEs son fundamentales para mantener la calidad del código.

Buenas Prácticas: Resumen Visual



Aplicar estas seis prácticas de forma sistemática reduce drásticamente la aparición de errores y hace tu código más legible para cualquier miembro del equipo.

Conclusión: Conviértete en un Mejor Programador

Rastreo efectivo

Usa `print()` y depuradores para seguir el flujo exacto de tu código e identificar problemas con precisión.

Evita errores comunes

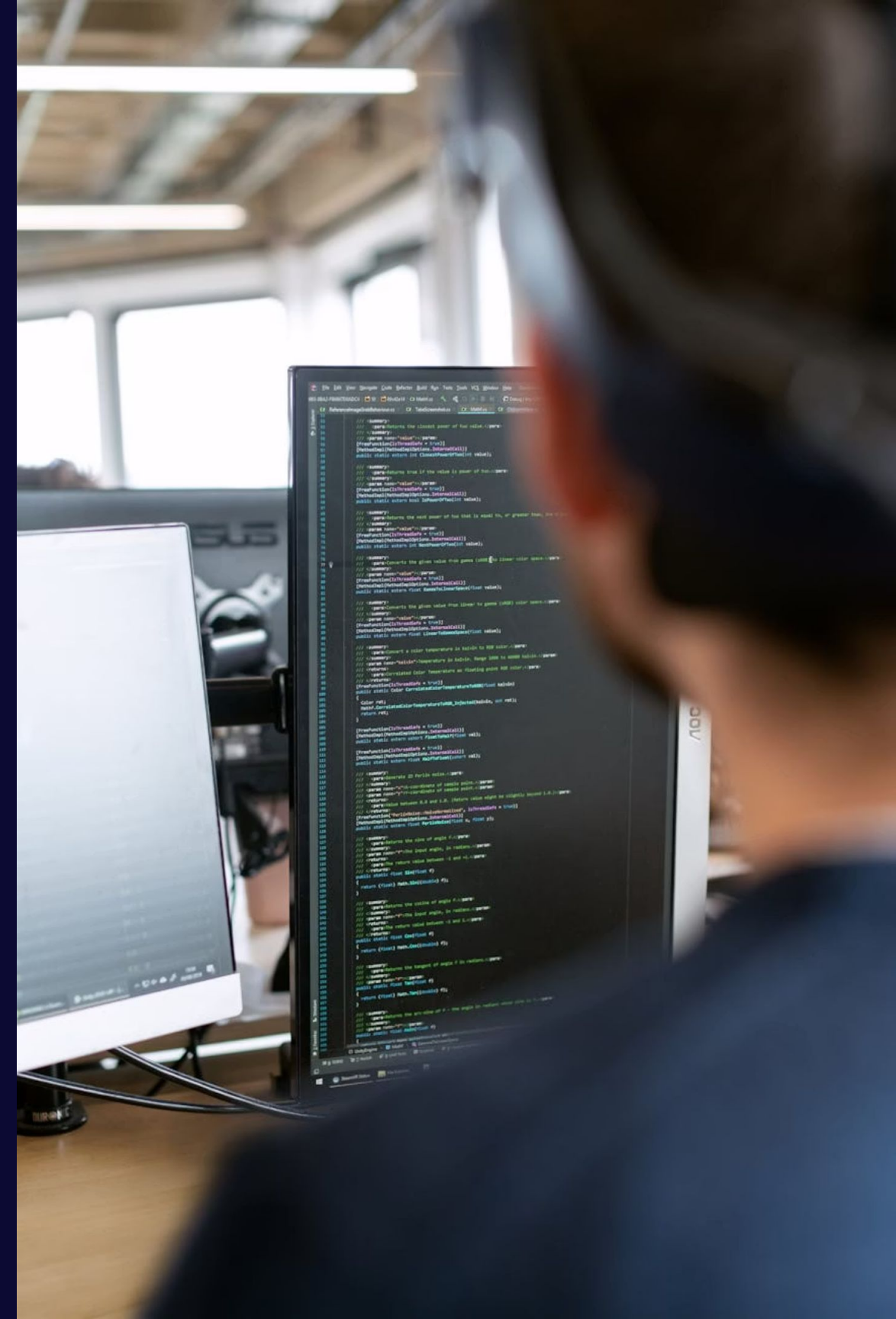
Indentación, ámbito, inmutabilidad y excepciones no manejadas son los culpables más frecuentes de fallos en Python.

Aplica buenas prácticas

Código limpio, bien nombrado y con manejo de errores es sinónimo de código profesional y mantenible a largo plazo.

Mejora continua

Pruebas automatizadas, depuradores avanzados y revisión de código elevan tus habilidades al siguiente nivel profesional.



Listas en Python: Domina los Datos desde Cero

Los datos reales son desordenados. Las listas son la herramienta fundamental de Python para

filtrar, transformar y automatizar

el trabajo con colecciones de información, desde registros de clientes hasta resultados científicos.



Finanzas

Calcular promedios de precios de acciones



Ciencia

Filtrar resultados experimentales



Marketing

Personalizar correos masivos



Automatización

Renombrar archivos y agrupar datos

Las listas en Python tienen tres atributos

❖ Orden

- Mantienen el orden en que los elementos fueron agregados.

❖ Mutabilidad

- Se pueden modificar después de su creación.

❖ Versatilidad

- Pueden almacenar diferentes tipos de datos, incluso otras listas.

Orden — Almacenamiento organizado

Las Listas en Python **recuerdan exactamente el orden** en que añadiste cada elemento. Esto no es trivial: significa que el primer elemento siempre estará en la posición 0, el segundo en la posición 1, y así sucesivamente, sin importar cuántas operaciones realices después.

¿Por qué importa el orden?

- Puedes acceder a cualquier elemento por su índice de forma inmediata.
- El orden de inserción se conserva siempre, sin reordenamientos automáticos.
- Puedes recorrer la lista de principio a fin con total predictibilidad.

Casos de uso ideales

- Pasos de un proceso secuencial
- Eventos ordenados cronológicamente
- Listas de tareas pendientes
- Registros en orden de llegada



Ejemplo rápido: `tareas = ["diseñar", "codificar", "probar"]`

el orden importa: siempre diseñamos antes de codificar.

Mutabilidad: Listas que Evolucionan con tus Datos

¿Qué significa mutable?

Una lista mutable puede modificarse **después de su creación**, a diferencia de las tuplas u otras estructuras fijas. Esto las convierte en la estructura ideal para datos que cambian con el tiempo.

- Agregar nuevos elementos
- Eliminar elementos existentes
- Cambiar valores en cualquier posición

Casos de uso reales

La mutabilidad es especialmente valiosa cuando trabajas con información dinámica:

Inventarios: añadir o eliminar productos en tiempo real

Registros de clientes: actualizar datos de contacto

Listas de tareas: marcar ítems como completados

Sesiones de usuario: rastrear acciones en una app

- 📌 Una lista es como una pizarra: puedes borrar y reescribir cuando necesites, sin tener que empezar de cero.

Versatilidad: Un Contenedor para Todo

Crear una lista en Python es tan sencillo como usar corchetes `[]`. Y Una de las grandes ventajas de las listas en Python es su capacidad de **almacenar cualquier tipo de dato**, incluso mezclados en la misma colección.

Cómo declarar listas

Lista vacía

```
new_list = []
```

Útil cuando aún no tienes datos pero quieres preparar la estructura.

Lista con elementos

```
grocery_list = ["milk", "hummus",  
               "bread", "cheese", "apples"]
```

La forma más directa: declaras y poblás la lista en la misma línea.

Tipos más comunes

Números primos

```
primes = [2, 3, 5, 7, 11, 13]
```

Resultados booleanos

```
coin_flips = [True, False, True, True]
```

Registro mixto

```
student_record = ["Aashvi", 24,  
                  "Computer Science", 3.8, True]
```

Operaciones Básicas: Acceder y Recortar Listas

Acceder a los elementos de una lista y extraer subconjuntos son habilidades esenciales. Python usa índices que empiezan en 0 y una notación de rebanado muy expresiva.

```
grocery_list = ["milk", "hummus", "bread", "cheese", "apples"]
```

1

Indexación

```
grocery_list[0]    # "milk"  
grocery_list[-1]   # "apples"  
len(grocery_list) # 5
```

El índice `-1` devuelve siempre el último elemento, sin importar el tamaño.

3

Slice con paso

```
grocery_list[0:5:2]  
# ["milk", "bread", "apples"]
```

El tercer parámetro define el salto entre elementos seleccionados.

2

Slicing (rebanado)

```
grocery_list[0:2]  
# ['milk', 'hummus']
```

Extrae un subconjunto. El índice final es **exclusivo**: `[0:2]` devuelve los índices 0 y 1.

4

Invertir la lista

```
grocery_list[::-1]  
# ["apples", "cheese", "bread",  
#   "hummus", "milk"]
```

Paso `-1` recorre la lista de derecha a izquierda.

Métodos, Comprensión y Listas 2D

Métodos más usados

Método	Función
<code>append(x)</code>	Agregar al final
<code>insert(i, x)</code>	Insertar en posición
<code>remove(x)</code>	Eliminar elemento
<code>sort()</code>	Ordenar
<code>pop(i)</code>	Eliminar y devolver
<code>x in lista</code>	Verificar existencia
<code>count(x)</code>	Contar ocurrencias

Listas bidimensionales

Perfectas para matrices, tableros de juego o tablas de datos.

```
grid = [
    [1,2,3],
    [4,5,6],
    [7,8,9]
]

print(grid[1][2])

salida :
6
```

Comprensión de listas

Crea listas nuevas de forma **concisa y eficiente** en una

```
[expression for item in iterable]
```

```
# Aplicar una operacion a una lista
numeros = [55, 70, 78, 52, 68]
dobles = [s ** 2 for s in exam_scores]
```

```
# Solo números pares
evens = [x for x in range(1,11)
         if x % 2 == 0]
```

```
# Par o impar
labels = ["par" if x%2==0 else "impar"
          for x in range(1, 6)]
```



Errores comunes: `IndexError` (índice fuera de rango)
`ValueError` (valor inexistente).

¿Lo has Entendido? Pon a Prueba tu Conocimiento

Dos preguntas prácticas para consolidar lo aprendido sobre listas en Python.

Pregunta 1

Estás creando un sistema para registrar las ventas diarias de una tienda durante un mes. ¿Qué característica hace que las listas sean la estructura MÁS adecuada para esta tarea?

✓ Respuesta correcta: Las listas **mantienen el orden** en que se añaden los elementos, conservando la secuencia cronológica de los datos de venta. Esto permite analizar tendencias a lo largo del tiempo de forma precisa.

Pregunta 2

Tienes una lista de clientes con historial de compras. ¿Qué técnica es MÁS eficaz para extraer solo aquellos que superaron un importe determinado?

✓ Respuesta correcta: Usar una **comprensión de lista con condicional**: `[c for c in clientes if c["compra"] > umbral]`. Es la forma más concisa, legible y eficiente para filtrar colecciones en Python.